

LESSON 01 • 执行基础

EVM、交易与状态变化

理解智能合约安全的第一步，是把合约看成一个由交易驱动的确定性状态机，而不是一段长期驻留、持续运行的服务器程序。

状态

认识链上持久状态与一次调用中的临时数据。

执行

解释交易、消息调用、Gas 与回滚之间的关系。

安全

从“谁能触发什么状态变化”开始审计。

核心原理：一笔交易要么把全部有效状态变化提交到链上，要么在异常时整体回滚；但它已经消耗的 Gas 不会退回。

确定性状态机

EVM 节点以相同的前置状态、交易输入和区块上下文执行同一份字节码，得到相同结果。合约不能直接请求互联网，也不能依赖本地随机数，否则不同节点无法达成共识。

交易输入 → 读取状态 → 执行字节码 → 提交新状态

调用上下文中的关键值

- **msg.sender**: 当前这一次调用的直接发起者，可能是用户，也可能是另一个合约。
- **msg.value**: 随当前调用发送的原生代币数量。
- **block.timestamp**: 出块时间，可用于宽松截止时间，不适合高价值随机性。
- **tx.origin**: 整条调用链最初的外部账户，不能用于授权。

一个最小状态转换

```
contract Counter {
    uint256 public count;

    function increment() external {
        count += 1;
    }

    function add(uint256 n) external {
        require(n <= 10, "too large");
        count += n;
    }
}
```

审计时先问三件事

1. 状态变量代表什么业务事实？
2. 哪些入口能修改它？
3. 入口的身份、数值和时序约束是否充分？

常见误区：“合约执行失败就没有成本。”实际上状态会回滚，但矿工或验证者已经完成计算，消耗的 Gas 仍由发送者承担。

LESSON 02 • 数据位置

Storage、Memory 与 Calldata

数据位置决定变量的生命周期、是否可修改、复制成本以及别名行为。很多“看起来修改了”的代码，实际只修改了内存副本。

Storage 持久保存于链上，写操作昂贵。	Memory 仅在当前调用中存在，可修改。	Calldata 只读外部输入，通常避免额外复制。
----------------------------------	---------------------------------	-------------------------------------

数据位置	生命周期	可修改性	典型用途
storage	跨交易持久化	可修改	状态变量、状态引用
memory	一次消息调用	可修改	临时数组、计算结果
calldata	一次外部调用	只读	外部函数的动态参数

引用与副本的差异

```
struct Profile {
    uint128 score;
    bool active;
}
mapping(address => Profile) profiles;

function activate() external {
    Profile storage p = profiles[msg.sender];
    p.active = true; // 写入链上状态
}

function preview() external view
returns (Profile memory)
{
    Profile memory p = profiles[msg.sender];
    p.score += 1; // 仅改内存副本
    return p;
}
```

存储布局为何影响安全

EVM 以 32 字节槽位组织 storage。较小的相邻值可能被打包进一个槽位，但动态数组和 mapping 的元素位置通过哈希计算。代理升级时，如果变量顺序或类型发生不兼容变化，旧数据会被新逻辑错误解释。

边界与拒绝服务
不要在会增长的 storage 数组上执行无界循环。数组足够大时，单笔交易可能永远无法在区块 Gas 限制内完成。

Gas 的直觉

- 读取 calldata 通常比先复制到 memory 更节省。
- storage 写入远比内存运算昂贵。
- 删除 storage 可能产生退款，但退款受协议上限约束。

核心原理：先判断数据应该“持久存在、临时可改还是只读输入”，再选择数据位置；不要为了语法方便而隐式复制大型动态数据。

常见误区：“从 mapping 读取出的 struct 默认就是原状态引用。”是否为引用取决于显式数据位置；复制到 memory 后再修改不会更新 storage。

LESSON 03 • 权限边界

访问控制与最小权限

访问控制不是添加一个 `onlyOwner` 就结束了。安全设计要明确角色、能力、授权周期、撤销路径和高风险操作的延迟。

身份

区分直接调用者、签名者与被委托角色。

能力

每个角色只获得完成任务所需的最小权限。

治理

为密钥丢失、轮换与紧急暂停设计路径。

核心原理：权限应附着在具体能力上，而不是笼统地把所有管理操作交给一个永久超级管理员。

两步所有权转移

```
contract TwoStepAdmin {
    address public owner;
    address public pendingOwner;

    modifier onlyOwner() {
        require(msg.sender == owner, "not owner"); _;
    }

    function proposeOwner(address next)
        external onlyOwner
    {
        require(next != address(0), "zero address");
        pendingOwner = next;
    }

    function acceptOwnership() external {
        require(msg.sender == pendingOwner, "not pending");
        owner = msg.sender;
        pendingOwner = address(0);
    }
}
```

两步流程防止管理员因输入错误地址而永久失去权限。新地址必须主动证明自己发起交易。

角色拆分示例

- **PAUSER**：只能暂停危险入口，不能转移资金。
- **OPERATOR**：执行日常参数更新，受上下限约束。
- **UPGRADER**：发起升级，但需时间锁后才能执行。
- **TREASURY**：管理资金，最好由多签控制。

危险模式

用 `tx.origin` 鉴权时，恶意合约可以诱导管理员调用自己，再以管理员的 `tx.origin` 调用目标合约。目标看到的最初账户仍是管理员，从而错误放行。

权限审计清单

1. 列出所有修改状态或转移资产的外部入口。
2. 为每个入口标明允许角色与参数边界。
3. 检查初始化函数能否被重复或抢先调用。
4. 检查角色能否撤销、轮换和恢复。

常见误区：“管理员操作可以不做输入校验。” 管理密钥也会误操作或被攻破；权限检查与参数约束解决的是不同风险，二者都需要。

LESSON 04 • 外部交互

重入与 Checks-Effects-Interactions

外部调用会把执行控制权交给未知代码。只要关键状态尚未更新，对方就可能在第一次调用结束前重新进入原函数。

条件	顺序	纵深
识别外部调用、未结算状态和可重入入口的组合。	先检查、再更新内部状态、最后与外部交互。	结合重入锁、拉取支付与跨函数分析。



有漏洞：先交互后更新

```
function withdraw() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "empty");

    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok, "transfer failed");

    balances[msg.sender] = 0; // 更新得太晚
}
```

接收者的 receive 或 fallback 可再次调用 withdraw。第二次检查时余额仍未清零，于是相同资金被重复提取。

修复：先更新后交互

```
function withdraw() external nonReentrant {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "empty");

    balances[msg.sender] = 0; // Effects

    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok, "transfer failed");
}
```

如果转账失败，require 会让整笔调用回滚，余额清零也会撤销。因此先更新状态不会让用户在失败时丢失余额。

不止是同一个函数

跨函数重入：函数 A 外部调用前保留了中间状态，攻击者回调函数 B，B 读取或利用了这个不一致状态。审计必须按共享状态变量梳理所有入口，不能只检查原函数是否有锁。

只读重入：某个 view 函数在中间状态暴露错误价格，其他协议在回调期间读取该价格并作出资产决策。即使 view 不写状态，也可能放大组合风险。

常见误区：“使用 transfer 限制 2300 Gas 就能永久防重入。” Gas 成本会随协议升级变化，而且依赖固定 stipend 会破坏兼容性；应以状态顺序和显式防护为主。

LESSON 05 • 调用语义

call、staticcall 与 delegatecall

低级调用提供灵活性，也绕过了许多类型系统保护。尤其是 `delegatecall`，执行的是对方代码，修改的却是调用方状态。

call 在目标合约上下文执行，可发送原生代币。	staticcall 只读执行，禁止最终状态写入。	delegatecall 保留调用方存储、地址和调用上下文。
------------------------------------	-------------------------------------	--

方式	执行代码	使用的 Storage	常见场景
call	目标地址	目标合约	通用外部调用、发送 ETH
staticcall	目标地址	不可写入	只读查询、接口兼容探测
delegatecall	实现地址	调用方合约	代理升级、代码复用

检查成功标志与返回数据

```
function safeInvoke(address target, bytes calldata data)
    external returns (bytes memory result)
{
    require(target.code.length > 0, "not contract");
    (bool ok, bytes memory out) = target.call(data);
    if (!ok) {
        assembly {
            revert(add(out, 32), mload(out))
        }
    }
    return out;
}
```

低级调用失败不会自动让当前函数 `revert`，而是返回 `false`。忽略成功标志可能让本地状态记录“操作成功”，实际外部动作却没有发生。

delegatecall 的信任边界

实现合约可以改写代理的任意存储槽，也能代表代理继续外部调用。因此实现地址本质上拥有代理资产和权限，升级权限必须视为最高风险能力。

- 调用者与实现必须采用兼容存储布局。
- 初始化逻辑必须防止重复调用。
- 实现地址必须是合约并通过代码审查。
- 升级应经过多签、时间锁和链上事件。

接口调用优先

目标接口已知时优先使用类型化接口。编译器会编码参数、解码返回值，并在不匹配时失败；低级调用应集中封装在明确的边界模块中。

核心原理：每次低级调用都要回答：执行谁的代码、修改谁的状态、失败如何传播、返回数据如何验证、对方能否回调。

常见误区：“目标地址有代码就一定安全。” `code.length` 只能排除部分错误地址，不能证明代码语义、升级状态或返回数据可信。

LESSON 06 • 资产流转

原生代币转账与 Pull Payment

把资金主动推送给一组接收者，会把系统可用性绑定到每个接收者的行为。拉取支付把失败隔离到单个用户自己的领取操作。

会计

用负债映射记录每个接收者可领取金额。

隔离

单个接收者失败不能阻塞所有参与者。

守恒

总负债不得无依据地超过合约可用资产。

危险：循环推送

```
function payAll() external {
  for (uint256 i; i < users.length; ++i) {
    (bool ok, ) = users[i].call{
      value: rewards[users[i]]
    }("");
    require(ok, "one user blocked all");
  }
}
```

- 用户数量增长后可能超过区块 Gas 限制。
- 一个拒收资金的合约能让整批支付回滚。
- 循环中的外部调用扩大重入攻击面。

推荐：按需领取

```
mapping(address => uint256) public claimable;

function claim() external nonReentrant {
  uint256 amount = claimable[msg.sender];
  require(amount != 0, "nothing to claim");

  claimable[msg.sender] = 0;
  (bool ok, ) = msg.sender.call{value: amount}("");
  require(ok, "payment failed");
}
```

状态先清零，失败则整体回滚。每个用户独立领取，其他人的领取不会被恶意接收者阻塞。

建立可检查的资金不变量

负债守恒： $\text{sum}(\text{claimable}[\text{user}]) \leq \text{address}(\text{this}).\text{balance}$ 。如果合约还有不可领取的准备金，应进一步拆分“用户负债”和“协议储备”。

状态守恒：一次领取成功后，用户负债减少量必须等于实际转出量；失败时二者都不能变化。

强制转入的例外

合约不能假设自身余额只会通过 `receive` 增加。其他机制可能把原生代币强制送入，因此业务会计应依赖明确记录的存款与负债，不应把 `address(this).balance` 直接当作全部用户存款之和。

常见误区：“把 `require(ok)` 去掉就不会阻塞。” 这样虽然继续循环，却可能在未支付的情况下错误清除负债；失败隔离需要重新设计支付模型，而不是忽略错误。

LESSON 07 • 数值安全

整数、精度、单位与舍入

Solidity 没有浮点数。金融计算通常使用放大的整数表示小数，运算顺序和舍入方向会直接决定谁承担误差。

边界 理解 Solidity 0.8+ 的溢出检查与 unchecked。	精度 先乘后除以减少整数截断造成的信息损失。	单位 在变量名和接口边界显式区分 wei、token 与 share。
---	----------------------------------	---

先除后乘会丢失精度

```
// 错误: amount / total 可能先变成 0
uint256 payout = amount / total * pool;

// 更合理: 保留更多有效位
uint256 payout = amount * pool / total;
```

即便先乘后除，也要考虑乘法中间值是否溢出。高价值协议常使用经过审计的 mulDiv 实现，以 512 位中间精度完成乘除。

舍入方向是一项业务规则

- 铸造用户份额时向下舍入，避免用户获得超额份额。
- 计算用户偿债金额时可能需要向上舍入，避免留下微小坏账。
- 任何有利于重复调用者的舍入误差都可能被批量放大。

单位标注与类型收窄

```
uint256 constant WAD = 1e18;

function feeWei(uint256 amountWei)
    internal pure returns (uint256)
{
    return amountWei * 30 / 10_000; // 0.30%
}
```

收窄转换
从 uint256 显式转换为 uint64 不会自动证明值可容纳。转换前必须检查 value <= type(uint64).max，否则高位被截断。

何时使用 unchecked

只有在边界已由逻辑证明、并且节省 Gas 确有意义时才使用。例如循环计数器在循环条件下不可能达到上限。应把 unchecked 区域缩到最小并写出不变量。

风险	表现	防护
精度截断	小额结果变为 0	先乘后除、固定精度、mulDiv
单位混淆	把 6 decimals 当作 18 decimals	边界归一化、变量名携带单位
舍入套利	拆分交易反复获取误差	定义一致的保守舍入方向

常见误区：“Solidity 0.8 已检查溢出，所以数值逻辑天然安全。” 编译器不会替你检查精度损失、单位混淆、经济不变量或显式类型收窄。

LESSON 08 • 失败语义

require、revert、assert 与自定义错误

失败处理既是安全边界，也是对调用者的可观察接口。错误类型应准确表达“输入不合法、权限不足、外部条件不满足还是内部不变量被破坏”。

require/revert 拒绝可预期的无效请求或业务状态。	assert 声明内部永远应成立的不变量。	Custom Error 以结构化参数降低部署和运行成本。
---	---------------------------------	---

结构化错误

```
error Unauthorized(address caller);
error InsufficientBalance(
    uint256 available,
    uint256 requested
);

function withdraw(uint256 amount) external {
    if (msg.sender != owner) {
        revert Unauthorized(msg.sender);
    }
    uint256 available = balances[msg.sender];
    if (amount > available) {
        revert InsufficientBalance(available, amount);
    }
    balances[msg.sender] = available - amount;
}
```

错误选择

机制	适合表达
revert	权限、参数、余额、状态机前置条件
assert	理论上不可被外部输入破坏的内部不变量
try/catch	隔离外部合约调用或合约创建失败

不要吞掉关键失败
try/catch 只说明你处理了异常，不代表业务已经完成。捕获后必须明确选择重试、记录待处理负债、降级，或让本次操作整体失败。

不变量比单点条件更强

单个 require 检查一次入口，而不变量描述所有可达状态必须满足的关系。例如资金池可以声明：“总份额为零时总资产也应为零”或“所有账户负债之和不得超过受托管资产”。测试工具可通过模糊输入反复调用不同入口，寻找破坏这些关系的序列。

核心原理：可由用户触发的无效情况使用可解释的 revert；只有真正代表程序内部错误的条件才使用 assert。不要依靠错误消息本身承担访问控制。

错误传播

高层接口调用在失败时会自动向上传播 revert。低级 call 只返回成功标志和原始返回数据，调用方必须检查并决定是否原样冒泡错误。错误传播策略应与事务原子性需求一致。

常见误区：“catch 住外部错误后继续执行更健壮。”如果后续代码把未完成的外部动作当作成功，系统只是把显式失败变成了隐蔽的状态不一致。

LESSON 09 • 外部数据

预言机、价格操纵与时间窗口

链上合约无法自行判断“真实市场价格”。预言机提供的是带来源、更新时间和聚合规则的数据，使用者必须验证这些假设是否匹配业务风险。

新鲜度

拒绝超过业务容忍窗口的旧价格。

完整性

验证轮次、符号、精度与合理范围。

抗操纵

避免把单区块、低流动性现货价直接用于借贷。

验证价格，而不是只读取

```
function readPrice() internal view
    returns (uint256 price)
{
    (, int256 answer, , uint256 updatedAt, ) =
        feed.latestRoundData();

    require(answer > 0, "invalid price");
    require(block.timestamp - updatedAt <= 1 hours,
        "stale price");

    return uint256(answer); // 还需统一 decimals
}
```

安全窗口不是固定答案。清算系统可能需要分钟级新鲜度，低风险报表可以接受更长延迟。窗口应来自业务最大可承受损失，而不是复制其他协议的常量。

闪电贷如何放大现货操纵

临时借入巨资 → 推高池内价格 → 按假价借款 → 归还闪电贷

如果协议使用单个低流动性 AMM 的当前储备计算抵押品价格，攻击者只需在同一交易内短暂改变储备。交易结束后市场恢复，但协议已发放无法覆盖的债务。

缓解策略

- 使用多来源聚合或可信数据网络。
- 采用时间加权平均价格并验证窗口流动性。
- 设置价格偏离、借款规模和速率上限。
- 异常时暂停新增风险，而非强制错误清算。

精度归一化

不同价格源与代币可能分别使用 8、6 或 18 位小数。所有数值进入核心公式前应转换到约定精度，并在变量名中表达单位，例如 priceUsd1e18。转换顺序同样要检查溢出和舍入方向。

常见误区：“使用知名预言机就无需检查。” 数据源本身可能正常，但目标市场暂停更新、轮次过旧、精度理解错误，或者该数据源根本不适合当前资产。

LESSON 10 • 链下授权

签名验证与重放保护

签名证明某个私钥同意了一段消息，但它不会自动限定这份同意在哪条链、哪个合约、哪个时间段、以什么顺序使用。

域隔离

把 chainId 和 verifyingContract 纳入签名域。

唯一性

使用 nonce 保证每份授权只执行一次。

时效

使用 deadline 限制签名长期暴露的风险。

核心原理：安全签名应绑定“谁授权、授权什么、给谁、在哪使用、何时过期、这是第几次授权”。缺少任何维度都可能产生重放或语义歧义。

一个简化的结构化授权

```
bytes32 constant ACTION_TYPEHASH = keccak256(
    "Action(address user,address target,uint256 value,uint256
    nonce,uint256 deadline)"
);

function execute(Action calldata a, bytes calldata sig)
    external
{
    require(block.timestamp <= a.deadline, "expired");
    require(a.nonce == nonces[a.user]++, "bad nonce");

    bytes32 structHash = keccak256(abi.encode(
        ACTION_TYPEHASH, a.user, a.target,
        a.value, a.nonce, a.deadline
    ));
    address signer = recover(_hashTypedDataV4(structHash),
    sig);
    require(signer == a.user, "bad signer");
}
```

为什么字段必须完整

- 没有 **nonce**：相同签名可重复执行。
- 没有 **deadline**：泄露的旧签名可能永久有效。
- 没有 **target**：授权可能被用于另一个接收者。
- 没有 **chainId**：可能跨链重放。
- 没有 **verifyingContract**：可能被另一个合约复用。

顺序也重要

示例中的 nonce 自增发生在外部动作之前。如果后续执行 revert，自增也会回滚；如果成功，旧签名立即失效。执行外部调用前应完成签名验证和 nonce 消耗。

合约钱包与签名可塑性

不能假设所有用户都是 EOA。智能合约钱包需要通过标准接口验证签名。对 ECDSA 应使用成熟库检查 s 值和 v 值，避免同一授权存在多种有效编码；不要手写未经审计的恢复逻辑。

常见误区：“恢复出的地址正确，签名就绝对安全。” 地址恢复只验证签名者；若消息编码存在碰撞、域不完整或 nonce 未消耗，正确签名仍可能被错误使用。

LESSON 11 • 可升级系统

代理合约、初始化与存储布局

代理把“状态地址”和“逻辑代码”分开：用户调用代理，代理通过 `delegatecall` 执行实现代码。升级能力提高可维护性，也引入永久管理权限。

结构 代理持有状态，实现合约提供逻辑。	初始化 构造函数不会在代理存储中执行。	兼容性 升级必须保持既有存储槽含义不变。
-------------------------------	-------------------------------	--------------------------------



初始化必须只能成功一次

```
contract VaultV1 {
    bool private initialized;
    address public owner;
    mapping(address => uint256) public balance;

    function initialize(address initialOwner)
        external
    {
        require(!initialized, "initialized");
        require(initialOwner != address(0), "zero");
        initialized = true;
        owner = initialOwner;
    }
}
```

代理部署后若没有原子初始化，攻击者可能抢先调用 `initialize` 成为 `owner`。实现合约本身也应禁用初始化，避免被接管后产生意外影响。

安全的布局扩展

V1 槽位	V2 合理变化
slot 0: initialized + owner	保持原定义与顺序
slot 1: balance mapping	保持 mapping 位置
无	在末尾追加新变量

不兼容变化

删除、重排或改变已有变量类型，会让旧槽位被按新含义读取。即使新代码能够编译，链上历史状态也可能立即损坏。

升级权的治理

- 多签控制升级发起者。
- 时间锁留出监控和退出窗口。
- 升级前自动比较存储布局。
- 链上记录实现地址与升级事件。

核心原理：代理安全不仅是代码安全，还包括升级密钥、部署顺序、初始化原子性、存储兼容性和用户对可变规则的预期。

常见误区：“新实现只增加了一个变量，所以一定兼容。” 如果变量插入在旧变量之前，后续所有槽位都会错位；新变量通常只能追加到布局末尾。

LESSON 12 • 安全交付

从威胁建模到上线监控

审计不是上线前的一次性活动。高质量安全流程从业务不变量开始，经过测试、部署验证和权限核对，最终落到持续监控与可演练的响应方案。

建模 明确资产、角色、信任边界、攻击收益和失败后果。	验证 用单元、模糊、不变量和分叉测试覆盖不同层次。	运行 监控关键事件、权限变化、价格异常和资金守恒。
--------------------------------------	-------------------------------------	-------------------------------------

四层测试策略

- 1. **单元测试**：覆盖权限、边界值、失败路径和事件。
- 2. **模糊测试**：让工具生成大量输入，寻找单函数或短序列中的异常。
- 3. **不变量测试**：随机组合多个入口，持续检查资产与会计关系。
- 4. **分叉测试**：在真实链状态副本上验证代币差异、预言机和协议组合。

```
// 示例不变量
function invariant_assetsCoverLiabilities() public view {
    assert(address(vault).balance >= vault.totalLiability());
}

function invariant_noZeroAdmin() public view {
    assert(vault.owner() != address(0));
}
```

上线前检查

- 编译器、依赖与部署参数固定并可复现。
- 合约源码验证，地址与 chainId 二次核对。
- 管理员、多签阈值、时间锁和角色已实测。
- 初始化与升级脚本在测试网完整演练。
- 暂停、恢复和资金迁移路径有书面步骤。

上线后监控

- 异常大额转账和短时间高频调用。
- 角色授予、所有权转移和实现升级。
- 预言机过期、偏离和流动性骤降。
- 资产、负债、份额之间的不变量偏差。

把发现转化为可执行修复

问题描述	不足	可执行表达
“这里可能重入”	缺少攻击路径	指出外部调用、未更新状态、回调入口与可损失资产
“管理员权限太大”	缺少能力边界	列出可执行动作、最大损失，并建议多签、时间锁或角色拆分

核心原理：安全结论必须连接“前置条件 → 攻击步骤 → 被破坏的不变量 → 影响 → 可验证修复”，才能被开发、测试和运营共同执行。

常见误区：“通过审计就代表以后安全。” 审计基于特定代码、配置和假设；升级、权限变化、外部协议和市场环境都可能改变风险。